CPSC-354 Report

Brandon Foley
Chapman University

November 9, 2025

Abstract

Contents

1 Introduction

2 Week by Week

2.1 Week 1

2.1.1 Homework

Here is my work on the MU Puzzle:

$$MIU \to MIUIU \to MIUIUIU \to \cdots$$

$$MI \to MII \to MIIII \to MUI \quad \Rightarrow \quad MUI$$

$$MUI \to MUIU$$

$$MUI \to MUIU \to MUIUIU \to MUIUIUIU \to MUUIU \to MUUIUIU \to \cdots$$

$$MUIIU \to MUIIUUIU \to MUIU$$

$$MUIIU \to MUUIU \to MUIU$$

$$MUIU \to MUUIU \to MUIUIU \to MUIUIUIU \to \cdots$$

$$MUIUIUIU \to MUIUIUIUIU \to MUIUIUIUIUIU \to \cdots$$

The MU Puzzle is unsolvable because it is impossible to reach the string `MU` starting from `MI` using the given rules. After a little research and thinking on my part it is the number of `I`'s that make this impossible: rule applications can double the count, add one, or remove three at a time, however, none of these operations ever reduce the number of `I`'s to exactly zero. Since `MU` has no `I`'s, it can never be derived.

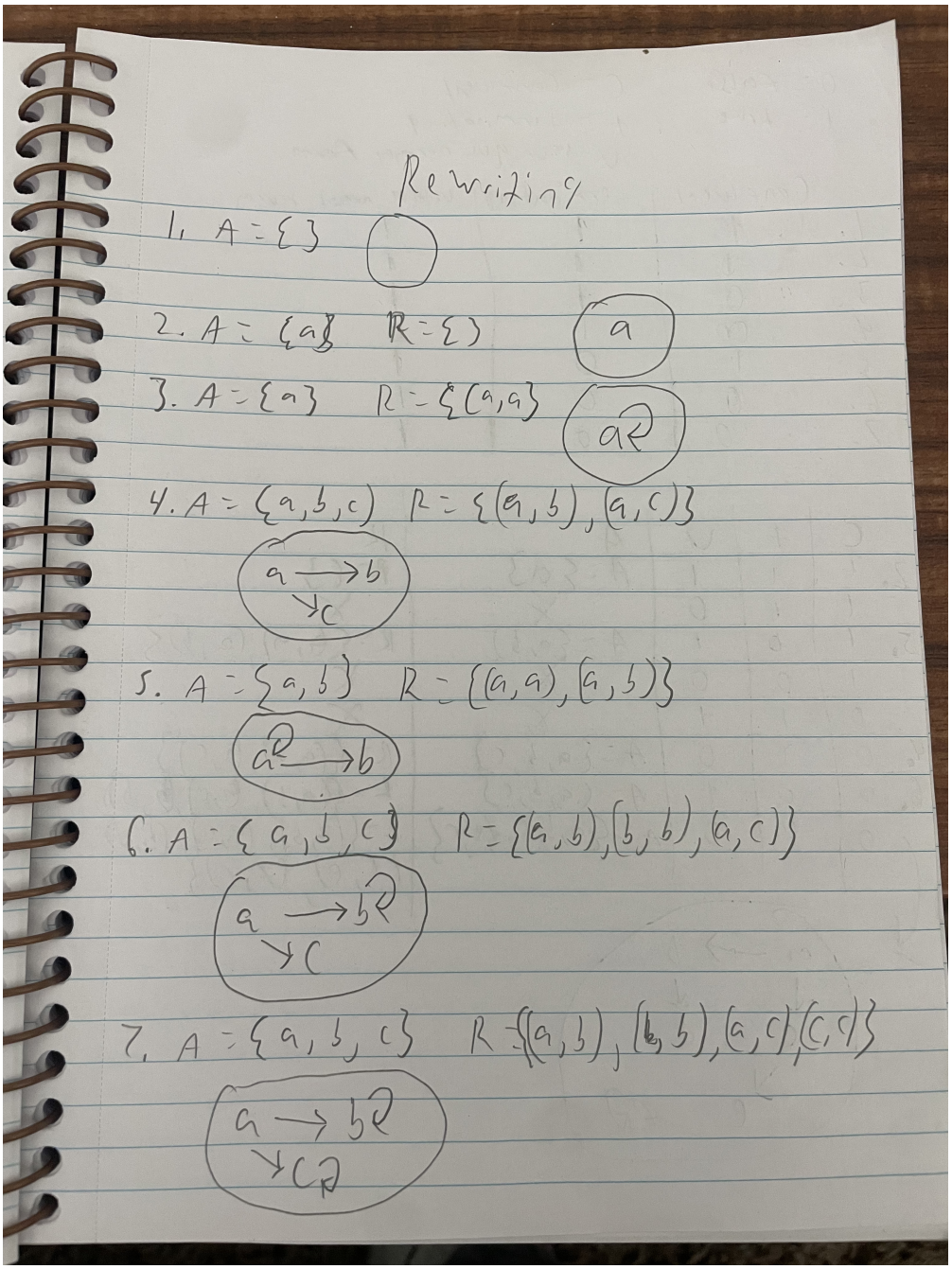
2.1.2 Questions

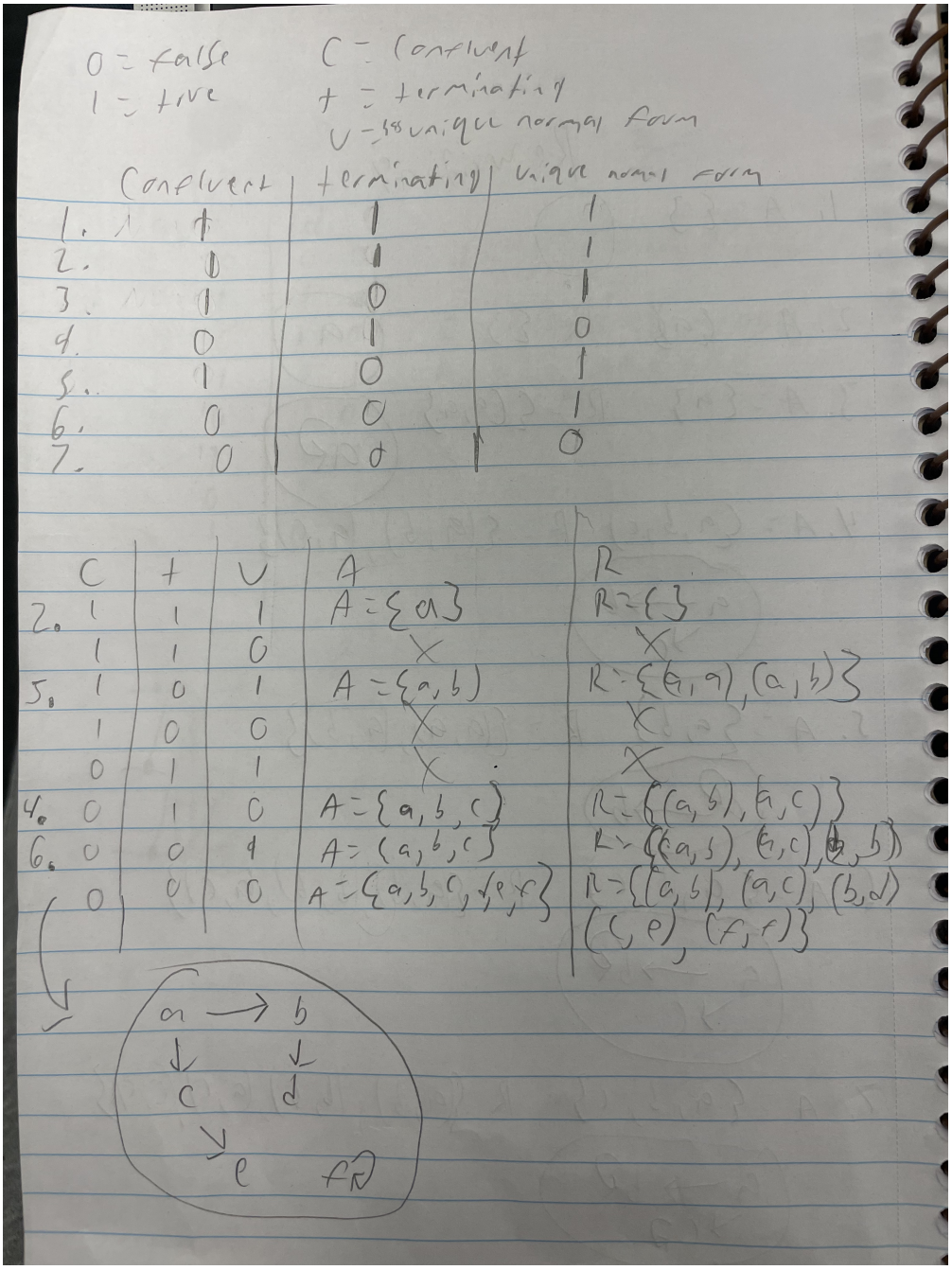
What rules could we add or remove in order to make this possible? I was thinking are there any ways to make this question solvable by adding only one more rule or maybe altering a rule. I think that this is an interesting question and concept.

2.2 Week 2

2.2.1 Homework

Here is my work on the Rewriting Homework:





In our homework we discovered 3 seperate cominations that are impossible to create. The first confluent, terminating and no unique normal form is impossible because if a system is confluent and terminating then it must have a unique normal form. The second confluent, not terminating and no unique normal form is impossible because if a system is confluent but not terminating then it must have a unique normal form when it exists. The third not confluent, terminating and unique normal form is impossible because if a system is terminating and has a unique normal form then it must be confluent.

2.2.2 Questions

When relating to programming, if a system is confluent but not terminating, like in some programs with infinite loops, how does confluence still guarantee that the result of a program is unique when it exists?

2.3 Week 3

2.3.1 Homework

Here is the work on ARS rewriting HW:

Exercise 5:

- Reduce some example strings such as `abba` and `bababa`.

$$abba \to baba \to baab \to bb \to b \to \{\}$$

$$bababa \to baabba \to bbba \to bba \to ba \to a$$

- Why is the ARS not terminating?
  The ARS is not terminating because you can infinitely go from ab to ba and vice versa.
- How many equivalence classes does $\leftrightarrow^*$ have? Can you describe them in a nice way? What are the normal forms?
  There are two equivalence classes: words with an even number of a's vs words with an odd number of a's.
  The normal forms are the empty string $\{\}$ and the string `a`.
- Can you change the rules so that the ARS becomes terminating without changing its equivalence classes?
  You can change $ba \to ab$ to $ba \to b$. This keeps the same equivalence classes but makes the system terminating.
- Write down a question or two about strings that can be answered using the ARS.
  - Does this string reduce to the empty string?
  - Are two given strings equivalent under the ARS?

Exercise 5b:

- Reduce some example strings such as `abba` and `bababa`.

$$abba \to baba \to baab \to bab \to ba \to a$$

$$bababa \to baabba \to babba \to baba \to aba \to aa \to a$$

- Why is the ARS not terminating?
  The ARS is not terminating because you can infinitely go from ab to ba and vice versa.
- How many equivalence classes does $\leftrightarrow^*$ have? Can you describe them in a nice way? What are the normal forms?
  There is one equivalence class: all words reduce to one a.
  The normal form is just `a`.

- Can you change the rules so that the ARS becomes terminating without changing its equivalence classes?
  You can change $ba \to ab$ to $ba \to b$. This keeps the same equivalence classes but makes the system terminating.
- Write down a question or two about strings that can be answered using the ARS.
  - Is there a unique normal form for this ARS?
  - Are two given strings equivalent under the ARS?

Bonus: Exercise 8

- Starting configuration:

$$\begin{aligned} &aaaaaaaaaaaaaaabbbbbbbbbbbbbbccccccccccccc \\ \to^* &\, acccccccccccccccccccccccccccccccccccccccc \\ \to &\, bbccccccccccccccccccccccccccccccccccccc \\ \to^* &\, bbbbbbbbccccccccccccccccccccccccccccccc \end{aligned}$$

- It is impossible to reach a configuration of only a's, only b's, or only c's.
- At each step, the rules correspond to the following change-vectors:

$$(a-1,\, b-1,\, c+2), \quad (a-1,\, b+2,\, c-1), \quad (a+2,\, b-1,\, c-1)$$

- Reducing these modulo 3 gives

$$(-1,\, -1,\, -1)$$

  for each move (since $+2 \equiv -1 \pmod 3$).
- Thus, we can simply subtract the numbers of a's, b's, and c's directly:

$$a - b = 15 - 14 = 1 \pmod 3$$

$$b - c = 14 - 13 = 1 \pmod 3$$

- Since these differences never reduce to 0, there will always be one leftover letter that prevents the string from reducing to a uniform string of only a's, b's, or c's.

Questions Given the rules for changing letters in Exercise 8 ARS, why is it impossible to end up with a string containing only one type of letter, and how can we use modular arithmetic to prove it?

2.4 Week 4

2.4.1 Homework

HW 4.1

Consider the following algorithm:

```
while b != 0:
    temp = b
    b = a mod b
    a = temp
return a
```

Under certain conditions this algorithm always terminates. In order for the algorithm to terminate:

- b >= 0
- a >= 0
- a and b must be integers, since doubles may cause unexpected mod results

A suitable measure function is:

$$m(a, b) = b$$

Proof of termination: Each iteration replaces b with $a \mod b$, which is always strictly smaller than the previous b but still nonnegative. Since b is a nonnegative integer, it cannot decrease indefinitely, so eventually $b = 0$ and the loop terminates.

HW 4.2

Consider the following fragment of an implementation of merge sort:

```
function merge_sort(arr, left, right):
    if left >= right:
        return
    mid = (left + right) / 2
    merge_sort(arr, left, mid)
    merge_sort(arr, mid+1, right)
    merge(arr, left, mid, right)
```

We define the measure function:

$$m(left, right) = right - left + 1$$

Proof:

- m maps to positive integers when $left \leq right$ since $m \geq 1$. If $left \geq right$ the function returns immediately (base case).
- If $left < right$ then $left \leq mid < right$, so both recursive calls shrink the measure:

  $$m(left, mid) = mid - left + 1 < right - left + 1 = m(left, right)$$

  $$m(mid + 1, right) = right - mid < right - left + 1 = m(left, right)$$

  Thus both recursive calls are on strictly smaller natural numbers.
- The natural numbers have no infinite strictly decreasing sequence, so repeated decreases of m force termination.

Therefore $m(left, right) = right - left + 1$ is a valid measure function for merge_sort: it is integer-valued, nonnegative, and strictly decreases on every recursive call.

2.4.2 Questions

Question: In a recursive backtracking algorithm, the procedure explores possible options and recurses until either a valid solution is found or all possibilities are exhausted. Since recursion can branch in many ways, how can we define a measure function that guarantees termination of the backtracking process, and what does this reveal about the relationship between the number of remaining choices and the maximum recursive depth?

2.5 Week 5

2.5.1 Homework

Here is the work on the lambda calculus evaluation:

$$(\lambda f.\, \lambda x.\, f(f(x)))\ (\lambda f.\, \lambda x.\, f(f(f\, x)))$$

Rename $f \mapsto g$ and $x \mapsto z$ in the second term:

$$(\lambda f.\, \lambda x.\, f(f(x)))\ (\lambda g.\, \lambda z.\, g(g(gz)))$$

$$\rightarrow\ (\lambda x.\, (\lambda g.\, \lambda z.\, g(g(gz)))\ ((\lambda g.\, \lambda z.\, g(g(gz)))\, x))$$

$$\rightarrow\ (\lambda x.\, (\lambda g.\, \lambda z.\, g(g(gz)))\ (\lambda z.\, x(x(xz))))$$

$$\rightarrow\ (\lambda x.\, \lambda z.\, ((\lambda z.\, x(x(xz))))(((\lambda z.\, x(x(xz))))(((\lambda z.\, x(x(xz))))z)))$$

$$\rightarrow\ (\lambda x.\, \lambda z.\, ((\lambda z.\, x(x(xz))))\ ((\lambda z.\, x(x(xz))))\ (x(x(xz))))$$

$$\rightarrow\ (\lambda x.\, \lambda z.\, ((\lambda z.\, x(x(xz))))\ (x(x(x(x(xz))))))$$

$$\rightarrow\ (\lambda x.\, \lambda z.\, x(x(x(x(x(x(xz)))))))$$

2.5.2 Questions

Why is -equivalence considered fundamental in the lambda calculus, and how does it affect the way we define substitution and reduction?

2.6 Week 6

2.6.1 Homework

Here is the step-by-step evaluation of the factorial function in λ-calculus:

$$\text{let rec fact} = \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * \text{fact}(n-1) \text{ in fact } 3$$

$$\rightarrow (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))\, 3$$

$$\rightarrow ((\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1))\ (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1))))\ 3$$

$$\rightarrow (\lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))(n-1)))\ 3$$

$$\rightarrow \text{if } 3 = 0 \text{ then } 1 \text{ else } 3 * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))(3-1))$$

$$\rightarrow 3 * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))\ 2)$$

$$\rightarrow 3 * ((\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1))\ (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1))))\ 2)$$

$$\rightarrow 3 * (\lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))(n-1)))\ 2$$

$$\rightarrow 3 * (\text{if } 2 = 0 \text{ then } 1 \text{ else } 2 * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))(2-1)))$$

$$\rightarrow 3 * 2 * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))\ 1)$$

$$\rightarrow 3 * 2 * (\lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))(n-1)))\ 1$$

$$\rightarrow 3 * 2 * (\text{if } 1 = 0 \text{ then } 1 \text{ else } 1 * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))(1-1)))$$

$$\rightarrow 3 * 2 * 1 * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))\ 0)$$

$$\rightarrow 3 * 2 * 1 * (\lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))(n-1)))\ 0$$

$$\rightarrow 3 * 2 * 1 * (\text{if } 0 = 0 \text{ then } 1 \text{ else } 0 * (\text{fix}(\lambda f.\, \lambda n.\, \text{if } n = 0 \text{ then } 1 \text{ else } n * f(n-1)))(0-1)))$$

$$\rightarrow 3 * 2 * 1 * 1 = 6$$

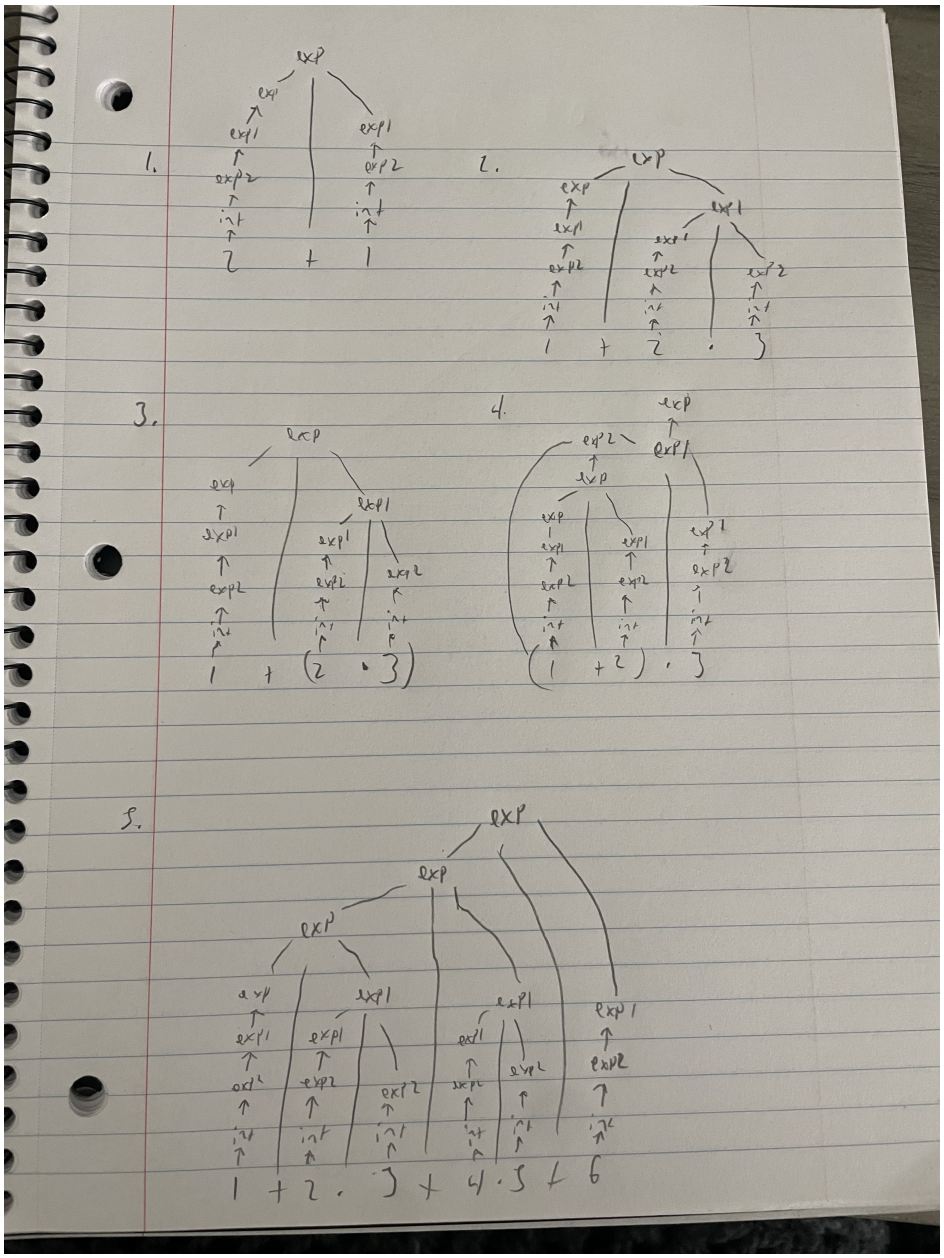
2.6.2 Questions

In this example, the fix operator allows us to define recursive behavior without explicit recursion syntax. How does the use of fix in λ-calculus compare to the use of recursion in functional programming languages, and why is this fixed-point construction needed for expressing general recursive functions in a purely functional system?

2.7 Week 7

2.7.1 Homework

Here is my homework for context free gramar trees:



2.7.2 Questions

In context-free grammars, how does introducing precedence levels (like Exp, Exp1, Exp2) formally enforce operator precedence compared to simply defining Exp -> Exp '+' Exp | Exp '*' Exp | Integer?

2.8 Week 8

2.8.1 Homework

Here is my work for the Lean proofs from the report.txt file:

Level 5:

```
rw [add_zero]
rw [add_zero]
rfl
```

Level 6:

```
rw [add_zero c]
rw [add_zero b]
rfl
```

Level 7:

```
rw [one_eq_succ_zero]
rw [add_succ]
rw [add_zero]
rfl
```

Level 8:

```
nth_rewrite 2 [two_eq_succ_one]
rw [add_succ]
rw [succ_eq_add_one]
nth_rewrite 1 [two_eq_succ_one]
rw [succ_eq_add_one]
rw [four_eq_succ_three]
nth_rewrite 1 [succ_eq_add_one]
rw [three_eq_succ_two]
rw [succ_eq_add_one]
rw [two_eq_succ_one]
rw [succ_eq_add_one]
rfl
```

2.8.2 Natural Language Proof (for Level 7)

In natural lanuguage we start with the statement that $1 + a = \text{succ}(a)$, where $\text{succ}(x)$ denotes the successor of x (that is, $x + 1$).

1. By definition, $1 = \text{succ}(0)$.
2. Substitute this into the left-hand side: $\text{succ}(0) + a$.
3. Using the recursive definition of addition, $\text{succ}(m) + n = \text{succ}(m + n)$. So, $\text{succ}(0) + a = \text{succ}(0 + a)$.
4. Next, by the base case of addition, $0 + a = a$.
5. Therefore, $\text{succ}(0 + a) = \text{succ}(a)$.
6. Hence, $1 + a = \text{succ}(a)$.

this shows that the proof follows directly from the base and inductive definitions of addition and the successor function in arithmetic.

2.8.3 Questions

When mathematicians informally rely on "obvious" or "intuitive" steps (like symmetry or associativity), how can such steps be represented in formal verification systems?

2.9 Week 9

2.9.1 Homework

Here is my work for the Lean induction proofs:

Exercise 1:

```
induction n
rw [add_zero]
rfl
rw [add_succ]
rw [n_ih]
rfl
```

Exercise 2:

```
induction b
repeat rw [add_zero]
rfl
repeat rw [add_succ]
rw [n_ih]
rfl
```

Exercise 3:

```
induction b
rw [add_zero]
rw [zero_add]
rfl
rw [add_succ]
rw [succ_add]
rw [n_ih]
rfl
```

Exercise 4:

```
induction c
repeat rw
[add_zero]
rfl repeat
rw [add_succ]
rw [n_ih]
rfl
```

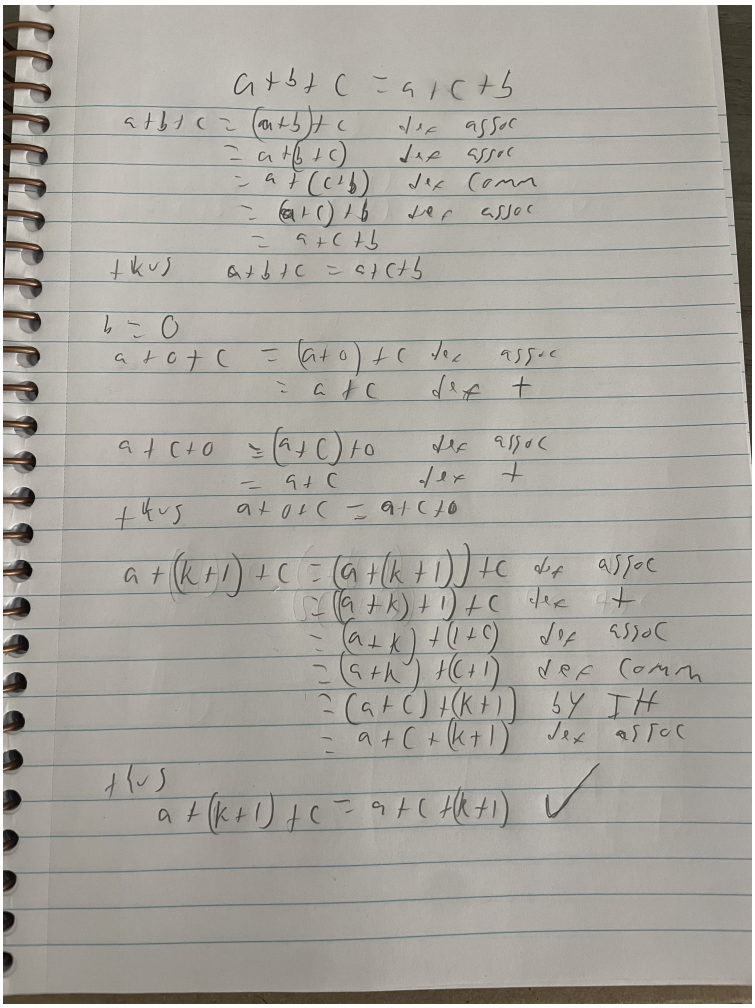
Exercise 5 (No Induction):

```
rw [add_assoc]
rw [add_assoc]
rw [add_comm b c]
rfl
```

Exercise 5 (Induction):

```
induction b
rw [add_zero]
rw [add_zero]
rfl rw [add_succ]
rw [succ_add]
rw [add_succ]
rw [n_ih]
rfl
```

Pen and paper proof for exercie 5:



2.9.2 Questions

When performing induction in Lean, why is it necessary to explicitly handle both the base and inductive cases, and how does Lean's treatment of recursive definitions ensure that each step of the induction corresponds to a structurally smaller argument?

2.10 Week 10

This week, we continued working in Lean, focusing on constructing proofs using `exact` statements and lambda expressions (λ).

1. **Exercise 6:**
$$\texttt{exact } \lambda c \mapsto \texttt{and\_intro } c \gg h$$
2. **Exercise 7:**
$$\texttt{exact } \lambda \langle c, d \rangle \mapsto h\ c\ d$$
3. **Exercise 8:**
$$\texttt{exact } \lambda (s : S) \mapsto \texttt{and\_intro } (h.\text{left } s)\ (h.\text{right } s)$$
4. **Exercise 9:**
$$\texttt{exact } \lambda (r : R) \mapsto \texttt{and\_intro } (\lambda (\_ : S) \mapsto r)\ (\lambda (\_ : \neg S) \mapsto r)$$

2.10.1 Questions

Question: How does Lean use λ-abstraction and `and_intro` to represent logical conjunction ($\wedge$) in proofs?

2.11 Week 11

This week, we continued working in Lean, focusing on constructing proofs using negation.

1. **Exercise 9:**
$$\texttt{exact } \lambda (a : P \wedge A) \mapsto h\ a.\text{left}\ a.\text{right}$$
2. **Exercise 10:**
$$\texttt{exact } \lambda (p : P)\ (a : A) \mapsto h\ (\texttt{and\_intro } p\ a)$$
3. **Exercise 11:**
$$\texttt{exact } \lambda a \mapsto h\ (\lambda n a \mapsto n a\ a)$$
4. **Exercise 12:**
$$\texttt{exact } \lambda n b \mapsto h\ (n b \gg \texttt{false\_elim})$$

2.11.1 Questions

Question: How can falsification be modeled within Lean's constructive logic, where the law of excluded middle does not hold, and what does this reveal about representing scientific reasoning in type theory?

3 Essay

4 Evidence of Participation

5 Conclusion

References

[BLA] Author, Title, Publisher, Year.